Guidelines - Time series

Claire Boyer & Olivier Coudray

université
PARIS-SACLAY

Table of contents

1. Introduction
2. Key concepts in time series analysis
3. Pre-processing for time series
4. Feature extraction
5. Models for time series
6. Best practices
7. Conclusion

What are Time Series?

Definition

A **time series** is a sequence of observations $(x_i)_{i=1}^N$ indexed by time:

- we denote t_i the timestamp of the ith observations
- the observations are ordered chronologically : $\forall i,\ t_i < t_{i+1}$
- $x_i \in \mathbb{R}$ (or $\mathbb{R}^d$ for multivariate series) represents the observed value at time t_i.

Classic framework: Most of the time, we handle time series that have a fixed acquisition frequency, such that:

$$t_i = t_0 + i\,\delta, \quad i = 1, \ldots, N,$$

where δ is the constant time interval between observations.

- **Univariate Time Series:** $x_i \in \mathbb{R}$, e.g., daily temperatures.
- **Multivariate Time Series:** $x_i \in \mathbb{R}^d$, e.g., meteorological data with temperature, humidity, and wind speed.

The specificities of time series

- Observations are not independent and identically distributed (i.i.d.).
- Temporal dependencies between observations.
- Common components:
  - **Trend:** Long-term increase or decrease.
  - **Seasonality:** Periodic fluctuations.
  - **Noise:** Random variation.
- Common properties:
  - stationarity
  - ergodicity

Machine learning tasks for time series

- **Forecasting:** Predict future values of the time series based on its past observations.

$$\hat{x}_{T+h} = f(x_1, \ldots, x_T), \quad h \geq 1.$$

  Examples: Weather forecasting, stock price prediction.
- **Timestamp-wise Prediction:** Assign a value or label to each timestamp:
  - **Classification:** Assign discrete labels, e.g., detecting activity type (walking, running).
  - **Regression:** Predict continuous values, e.g., power consumption at each timestamp.
- **Sequence-based Tasks:** Analyze entire time series (or segments) as single entities:
  - **Classification:** Assign a label to the entire curve, e.g., distinguishing between healthy and diseased ECG signals.
  - **Regression:** Predict a continuous outcome, e.g., age of a patient based on his/her ECG.
  - **Clustering:** Group similar time series based on patterns, e.g., customer behavior segmentation.
- **Other tasks:** anomaly detection, change point detection, imputation...

Forecasting

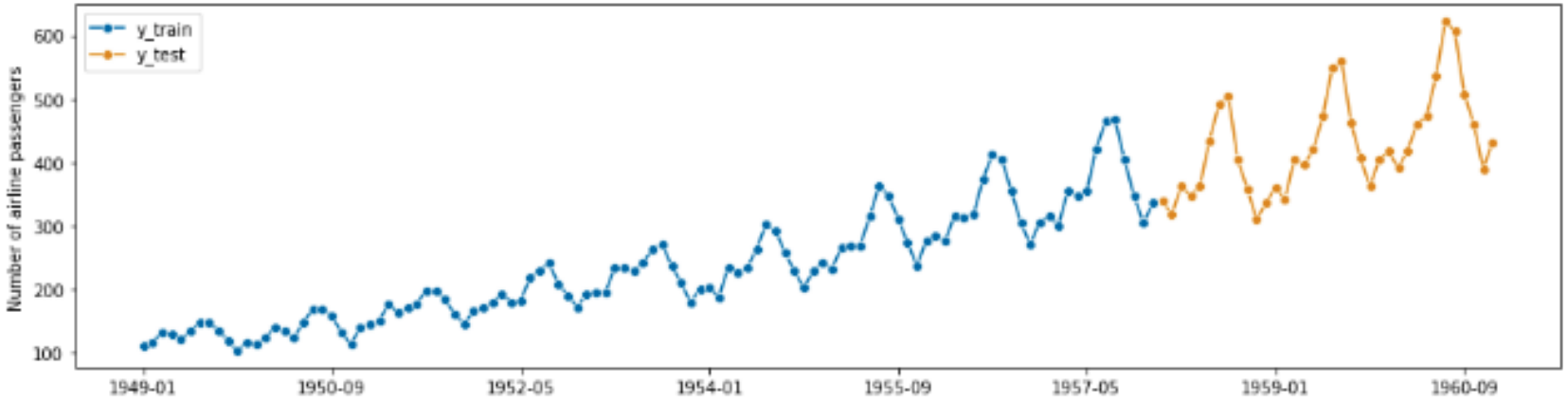

Figure: Airline dataset

Anomaly detection

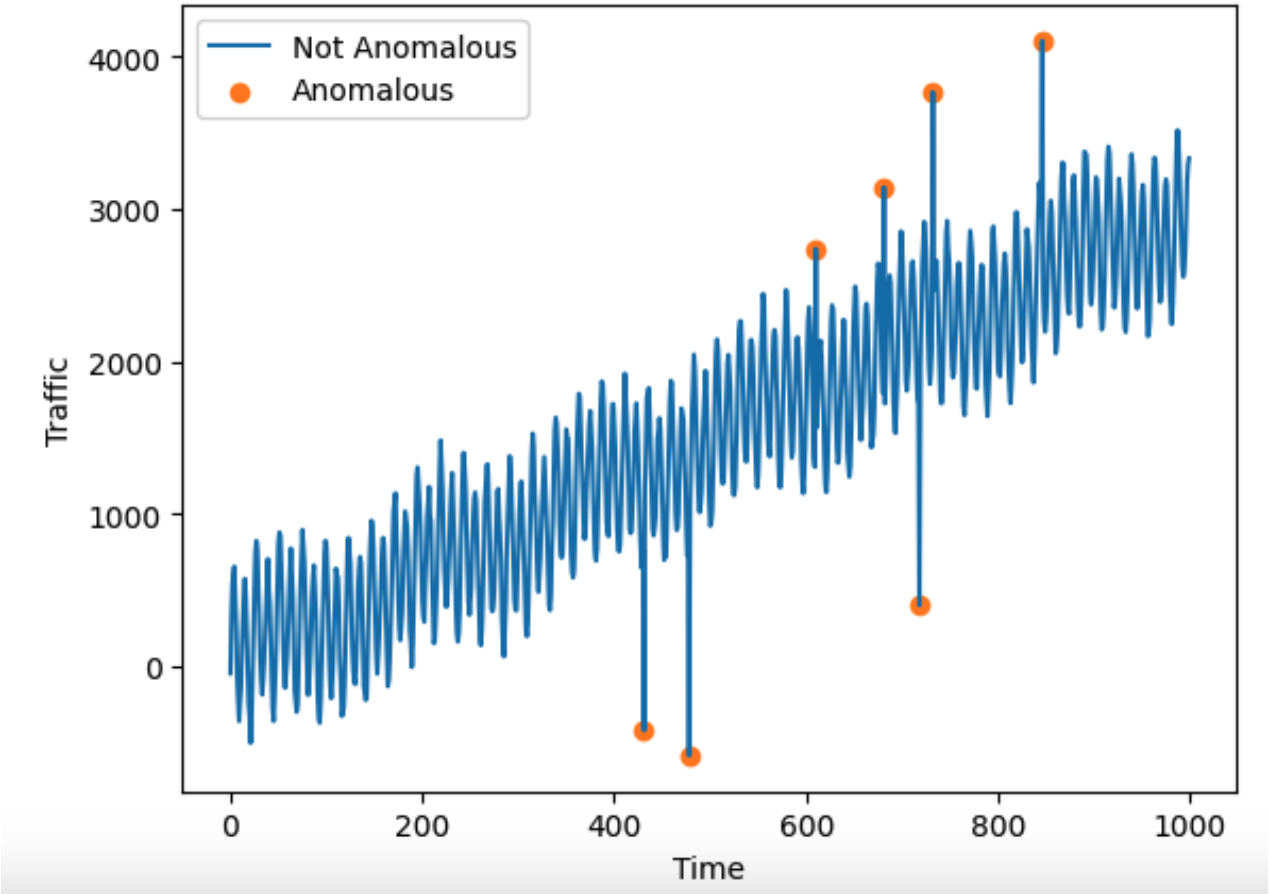

Figure: Yahoo dataset

Change-point detection

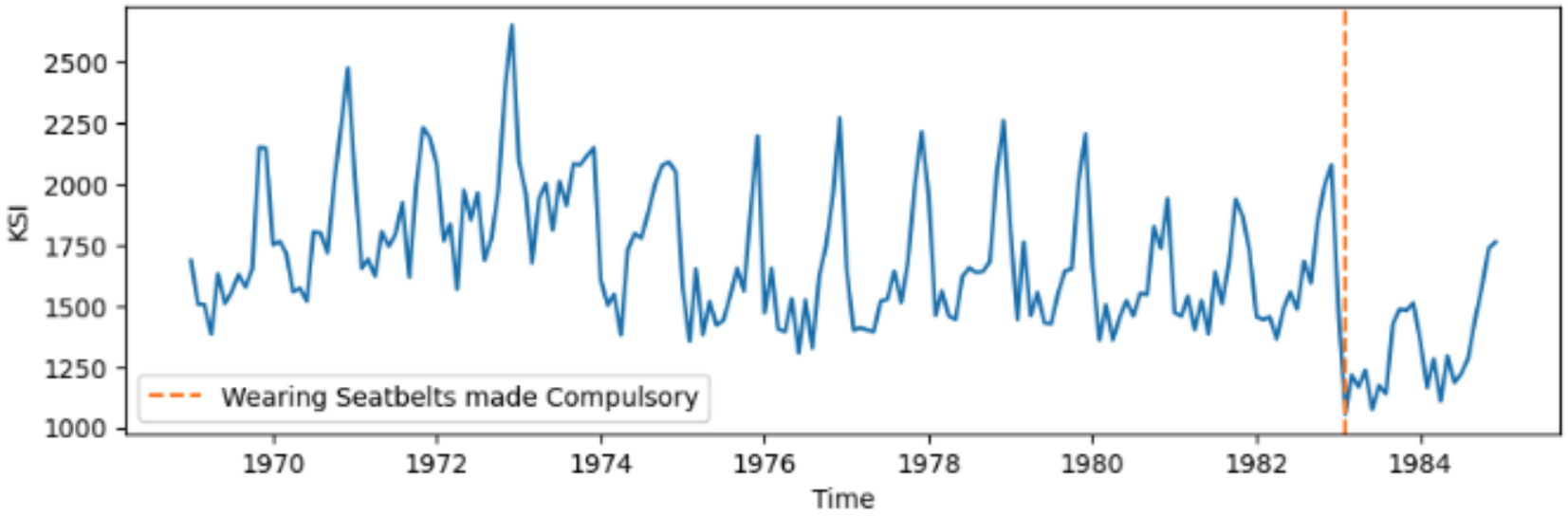

Figure: *Seatbelts* dataset

Overview of the Course

1. **Introduction:** Characteristics and importance of time series.
2. **Key concepts in time series analysis:** stationarity, ergodicity, auto-correlation.
3. **Preprocessing:** Handling missing values, decomposition.
4. **Feature Extraction:** Lagged features, rolling stats, spectral features.
5. **Modeling:** get back to traditional supervised learning, overview of classic time series models.
6. **Best Practices:** Avoiding pitfalls, practical guidelines.

Table of contents

1. Introduction
2. Key concepts in time series analysis
3. Pre-processing for time series
4. Feature extraction
5. Models for time series
6. Best practices
7. Conclusion

Stationarity in Time Series

Definition: A time series $\{X_i\}_{1 \leq i \leq N}$ is **stationary** if its statistical properties do not change over time.

Mathematically: A time series is stationary if for all $i \in \{1, ..., n\}$:

1. **Constant mean:**
$$\mathbb{E}[X_i] = \mu, \quad \forall i.$$
2. **Constant variance:**
$$\mathrm{Var}(X_i) = \sigma^2, \quad \forall i.$$
3. **Time-Invariant autocovariance:**
$$\mathrm{Cov}(X_i, X_{i+h}) = \gamma(h), \quad \forall i, h.$$

Stationarity in Time Series

Types of Stationarity:

- **Strict Stationarity:** The joint distribution of $\{X_{i_1}, X_{i_2}, \ldots, X_{t_k}\}$ depends only on the time differences $t_i - t_j$.
- **Weak Stationarity (Second-order Stationarity):** Only the mean, variance, and autocovariance are invariant over time.

Why Stationarity Matters:

- Many time series models assume stationarity for simplicity and mathematical tractability.
- Enables meaningful analysis of statistical quantities.

Checking stationarity in practice:

- Visualization : visible trend / seasonality, check autocorrelation function
- Statistical tests : ADF, KPSS

Autocorrelation Function

Autocorrelation Function

The **autocorrelation function** (ACF) of a time series measures the correlation between a time series and a lagged version of itself. For a stationary process X, the autocorrelation at lag h is defined as:

$$\rho(h) = \frac{\mathrm{Cov}(X_i, X_{i+h})}{\mathrm{Var}(X_i)} = \frac{\mathbb{E}[(X_i - \mu)(X_{i+h} - \mu)]}{\sigma^2},$$

where:

- $\mu = \mathbb{E}[X_i]$ is the mean of the process,
- σ^2 is the variance of the process,
- The ACF characterizes the dependence structure between observations at different lags.
- It only makes sense for **stationary** processes, where the statistical properties do not change over time.

Auto-correlation function

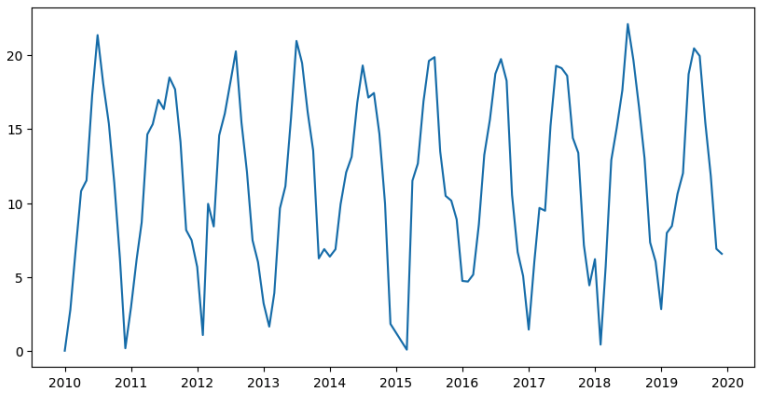

Figure: Monthly temperatures (Paris)

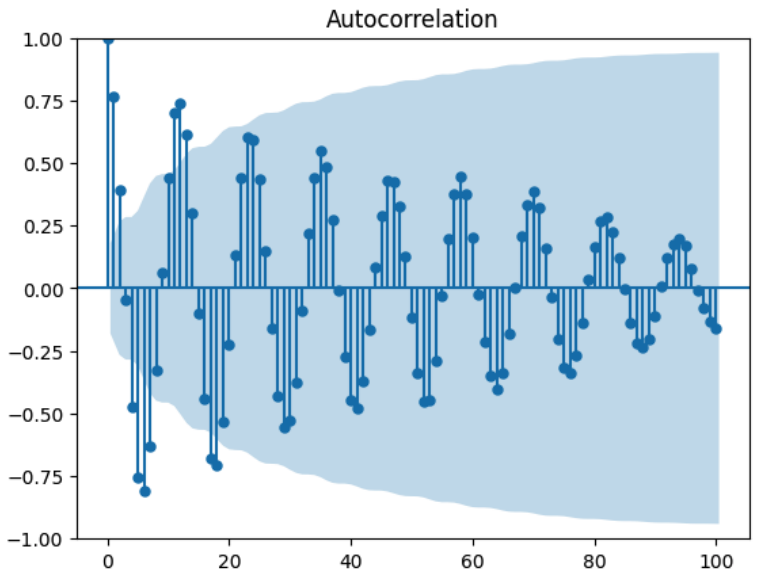

Figure: Autocorrelation

$\Rightarrow$ suggest the presence of a seasonal component.

Auto-correlation function

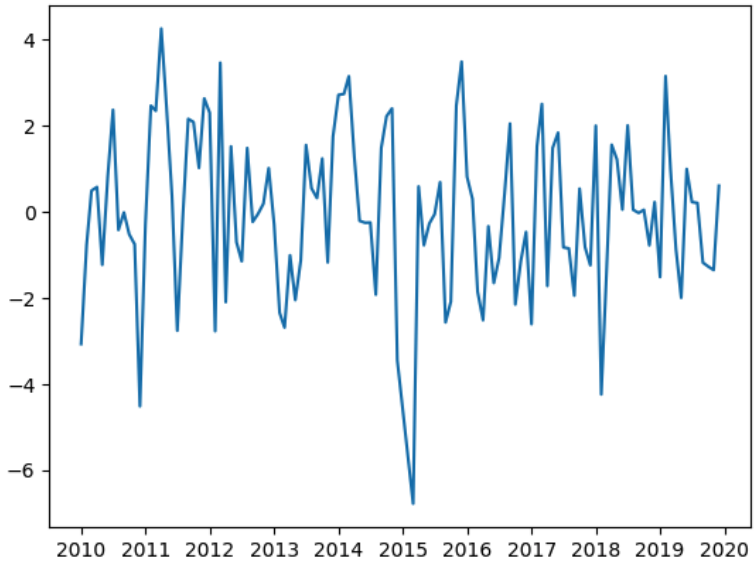

Figure: Monthly temperatures (Paris) after removing trend and seasonality

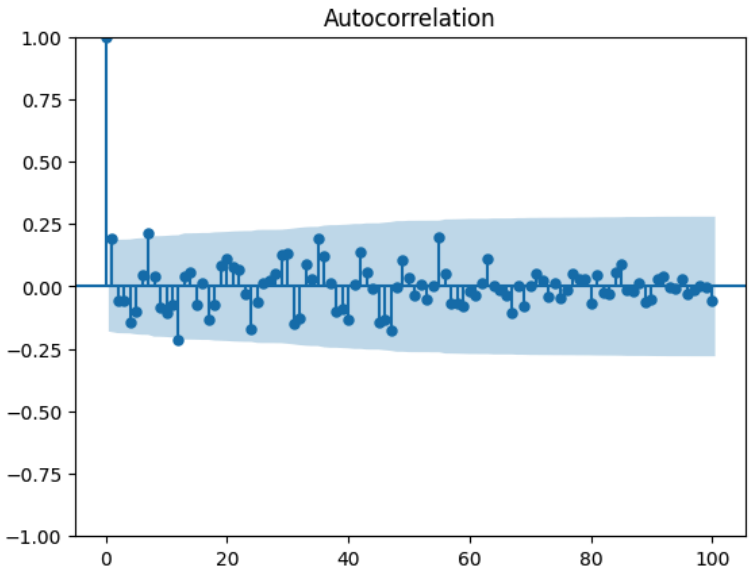

Figure: Autocorrelation

Ergodicity in Time Series

Ergodicity

A time series is said to be **ergodic** if its statistical properties can be inferred from a single, sufficiently long realization (i.e., the time series itself), rather than requiring multiple independent samples.

Mathematically, a process (X_i) is ergodic if:

$$\mathbb{E}[f(X_i)] = \lim_{N\to\infty} \frac{1}{N} \sum_{i=1}^{N} f(X_i),$$

for any measurable function $f(\cdot)$, where the expectation $\mathbb{E}$ is taken over the distribution of the process.

- This allows us to estimate statistical properties from a single time series realization.

Remarks and roles of stationarity and ergodicity

Remarks:

- there is no sense in studying mean, variance or auto-correlation function if the process is not stationary because these quantities may be time-dependent
- we cannot have ergodicity without stationarity

Role of stationarity and ergodicity :

- Stationarity: the statistical properties of the observations are constant over time
- Ergodicity: the statistical properties of the observations can be estimated from a single realization of the time series (do not require average over multiple realizations)

$\Rightarrow$ We can work with temporal mean / variance and the auto-correlation function

Table of contents

1. Introduction

2. Key concepts in time series analysis

3. Pre-processing for time series

4. Feature extraction

5. Models for time series

6. Best practices

7. Conclusion

Preprocessing for time series

Challenges to be addressed:

- dealing with irregular intervals between observations
- handling missing data
- identifying key components (trend, seasonality)
- ensuring stationarity

Irregular timestamps in time series

Irregular timestamps: observations are not necessarily uniformly paced (e.g., missing hours, irregular gaps)

Issues:

- hard to apply classic time series models
- trend and seasonality are harder to identify

Solutions:

- resampling: aggregating data to ensure regular intervals (mean, median, ...)
- interpolation: filling gaps (cf missing data)

Handling missing data for time series

Type of missing data (more on that in course 3 and 4):

- random sensor failure
- missing data in specific conditions : in a nuclear power plant, a sensor may stop recording when the power plant is off
- missing data related to extreme value: missing value when the range of acquisition of a sensor is exceeded

Techniques for imputing missing data in time series:

- expert knowledge
- forward fill / backward fill
- interpolation (linear, cubic,...)
- model-based imputation (ARIMA...)

Trend and Seasonality in Time Series

Trend

The **trend** is the long-term direction of a time series, either upward, downward, or flat. It can be modeled as:

$$X_i = T_i + \epsilon_i,$$

where T_i is the trend and ϵ_i is random noise. Trends can be linear, exponential, etc.

Seasonality

Seasonality refers to periodic fluctuations in a time series, repeating over a fixed period T. It can be modeled as:

$$X_i = S_i + \epsilon_i, \quad S_i = S_{i+T}.$$

Examples include annual sales cycles or weather patterns.

Trend and Seasonality in Time Series

Trend + Seasonality

A time series often includes both components:

$$X_i = T_i + S_i + R_i,$$

where R_i is the residual or noise.

Parametric estimation for trend and seasonality

Parametric model:

$$X_i = \sum_{k=1}^{p} \alpha_j \; g_k(t_i) + \sum_{k=p+1}^{q+p} \alpha_k \; g_k(t_i) + \varepsilon_i \; .$$

- g_1, ..., g_p and $g_{p+1}, ..., g_{p+q}$ are known function modeling respectively the trend and seasonality
- $\alpha_1, ..., \alpha_{p+q}$ are unknown coefficients $(p + q < N)$
- $(\varepsilon)_{1 \leq i \leq N}$ is a gaussian noise

Estimated components:

- Trend component:

$$\widehat{T}_i = \sum_{k=1}^{p} \widehat{\alpha}_j \; g_k(t_i)$$

- Seasonal component:

$$\widehat{S}_i = \sum_{k=p+1}^{q+p} \widehat{\alpha}_k \; g_k(t_i) \; .$$

Parametric estimation for trend and seasonality

Estimation using least square estimate:

Let:

$$K = \begin{pmatrix} g_1(t_1) & g_2(t_1) & \dots & g_{p+q}(t_1) \\ g_1(t_2) & g_2(t_2) & \dots & g_{p+q}(t_2) \\ \dots & \dots & \dots & \dots \\ g_1(t_N) & g_2(t_N) & \dots & g_{p+q}(t_N) \end{pmatrix}$$

Then:

$$\widehat{\alpha} = (K^T K)^{-1} K^T X$$

STL decomposition (example)

Dataset: monthly temperatures in Paris between 2010 and 2020

Choice of basis functions:

- Trend: $g_1(t) = 1$ (intercept), $g_2(t) = t$ (linear term), $g_3(t) = t^2$ (quadratic term)
- Seasonality: $g_4(t) = \cos(2\pi \frac{t}{T})$, $g_5(t) = \sin(2\pi \frac{t}{T})$, $g_6(t) = \cos(4\pi \frac{t}{T})$, $g_7(t) = \sin(4\pi \frac{t}{T})$ where $T = 12$ (annual periodicity)

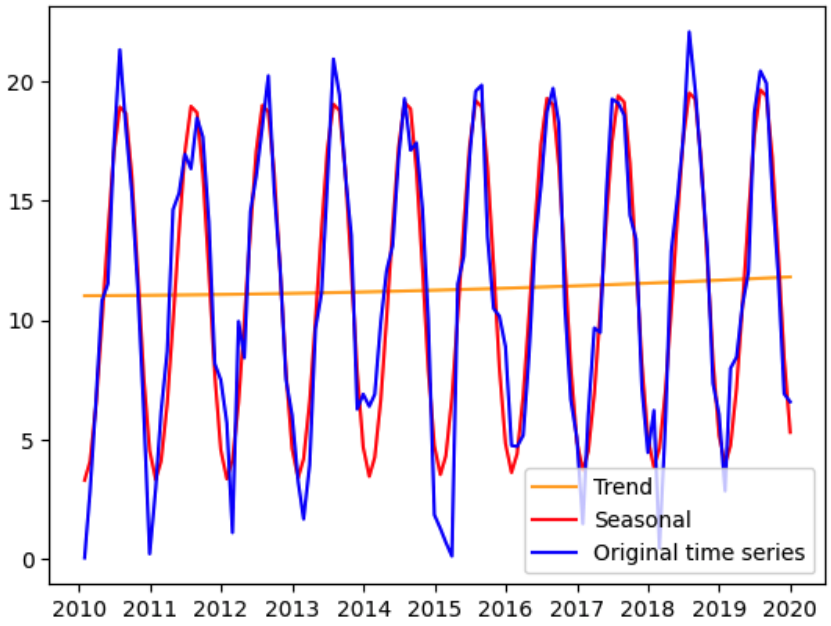

Figure: STL decomposition on temperature data

Ensuring stationarity

Checking stationarity in practice:

- Visualization : visible trend / seasonality, check autocorrelation function
- Statistical tests : ADF (Augmented Dickey-Fuller Unit Root Test), KPSS (Kwiatkowski-Phillips-Schmidt-Shin Test)

Remark on the hypotheses:

- ADF $\rightarrow \mathcal{H}_0$: non-stationarity vs $\mathcal{H}_1$: stationarity
- KPSS $\rightarrow \mathcal{H}_0$: stationarity vs $\mathcal{H}_1$: non-stationarity

Non-stationarity example

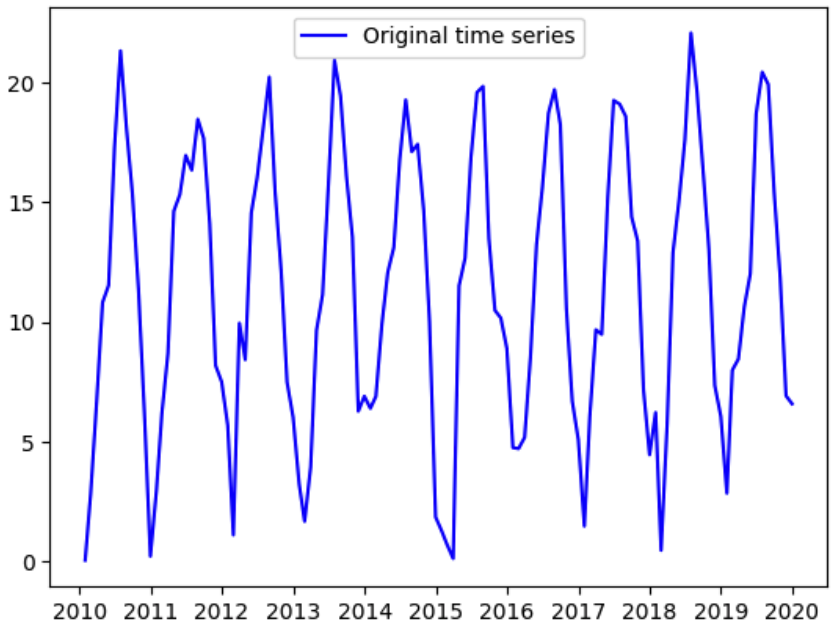

Figure: Monthly temperatures in Paris

- ADF test $\rightarrow p_{val} = 0.097$ (cannot reject $\mathcal{H}_0$ at level 5%)
- KPSS test $\rightarrow p_{val} = 0.033$ (reject $\mathcal{H}_0$ at level 5%)

Stationarity example

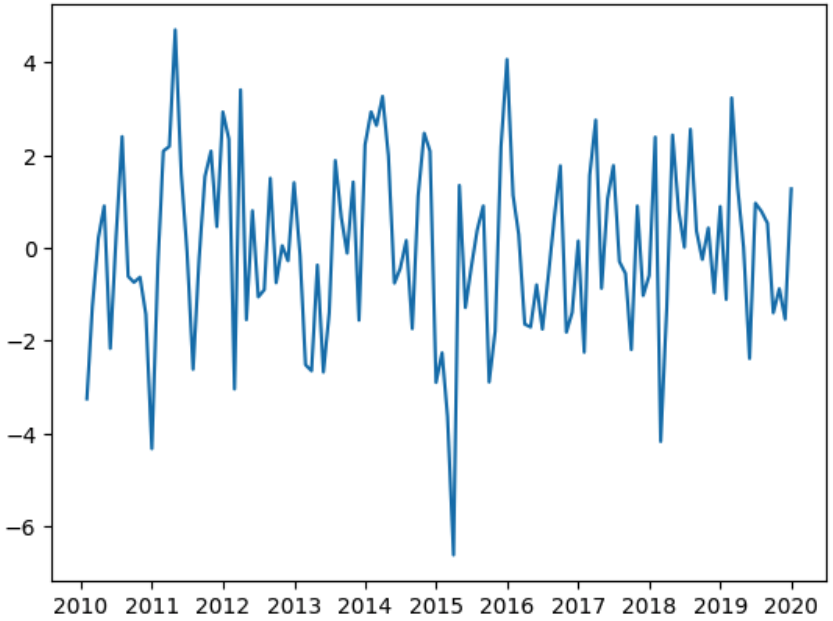

Figure: Monthly temperatures in Paris (after removing trend and seasonal components)

- ADF test $\rightarrow p_{val} \sim 10^{-15}$ (reject $\mathcal{H}_0$ at level 5%)
- KPSS test $\rightarrow p_{val} = 0.1$ (cannot reject $\mathcal{H}_0$ at level 5%)

Dealing with non-stationary data

Issue: Most methods in time series rely on the stationarity assumption.

How to deal with non-stationary data ?

- Look for trend and seasonal components to be removed from the original time series (cf STL decomposition)
- Differentiating :
  - $X'_t = X_t - X_{t-1} \rightarrow$ remove linear trend
  - $X''_t = X'_t - X'_{t-T}$ (T is the seasonal periodicity)

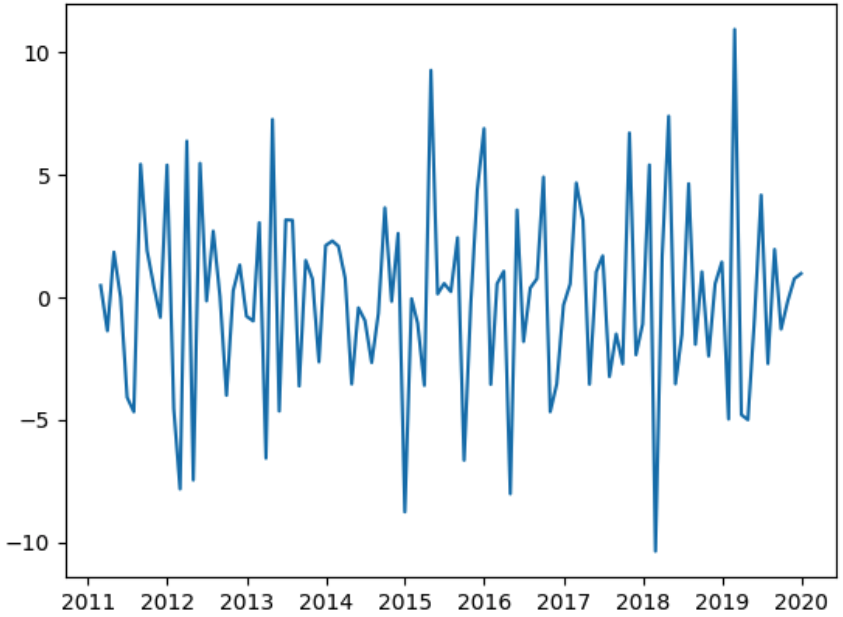

Figure: Monthly temperatures in Paris (after differentiating)

Table of contents

1. Introduction

2. Key concepts in time series analysis

3. Pre-processing for time series

4. Feature extraction

5. Models for time series

6. Best practices

7. Conclusion

Feature extraction for time series

Feature extraction:

- a feature is a real value that accounts for a certain property of the signal
- before addressing any machine learning task, we need to extract relevant features from time series

Why?

- reduce dimensionality: under its raw format, a time series is a high-dimensional vector whose main characteristics can usually be summarized in more compact representations
- improve data quality (smoothing...)
- enable compatibility with machine learning model $\rightarrow$ usually require tabular data
  - for sequence-level tasks (eg classification of time series), summarize each time series with a collection of features
  - for timestamp-level tasks (eg forecasting), include relevant information for modeling (calendar features, lags...)

Types of features

- Domain-specific feature extraction $\rightarrow$ e.g. extract relevant calendar information from datetime (time of day, day, month, year, week-end, holiday...)
- Statistic features $\rightarrow$ mean, median, variance, quantiles...
- Temporal features $\rightarrow$ lags, rolling statistics...
- Frequency domain features $\rightarrow$ FFT, spectrogram...
- Features extracted through deep learning (representation learning) $\rightarrow$ auto-encoders...

Domain-specific feature engineering

Examples of features:

- Relevant calendar features: year, month, day, hour, minute, time of day, day of week ($0 \rightarrow 7$), day of year ($0 \rightarrow 366$), week of year ($0 \rightarrow 52$), holiday, etc.
- Domain-related features $\rightarrow$ expert knowledge

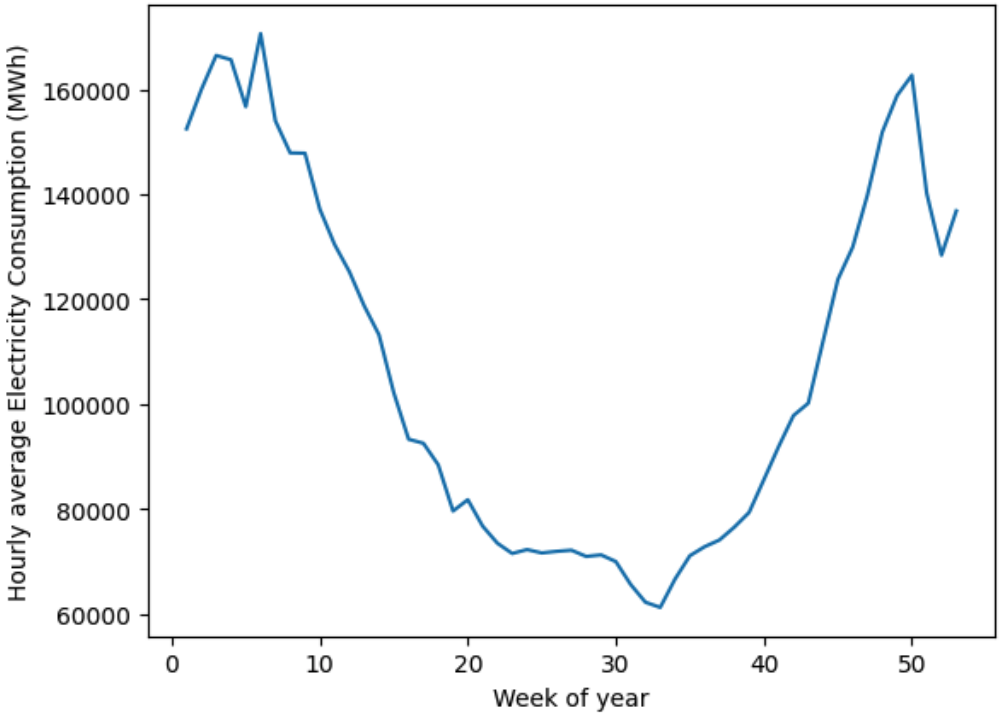

Figure: Electricity consumption in France (RTE)

Statistical features to summarize signals

Principle:

- compute summary statistics over each signals (or segments of signals)
- examples of statistics: mean, median, standard deviation, minimum, maximum, quantiles, time spent in a given range of values...

Remarks:

- only use statistics that are meaningful regarding the task
- these techniques can generate a large number of features among which:
  - some may be highly correlated
  - some may be difficult to interpret
- always start small

Statistical features: example

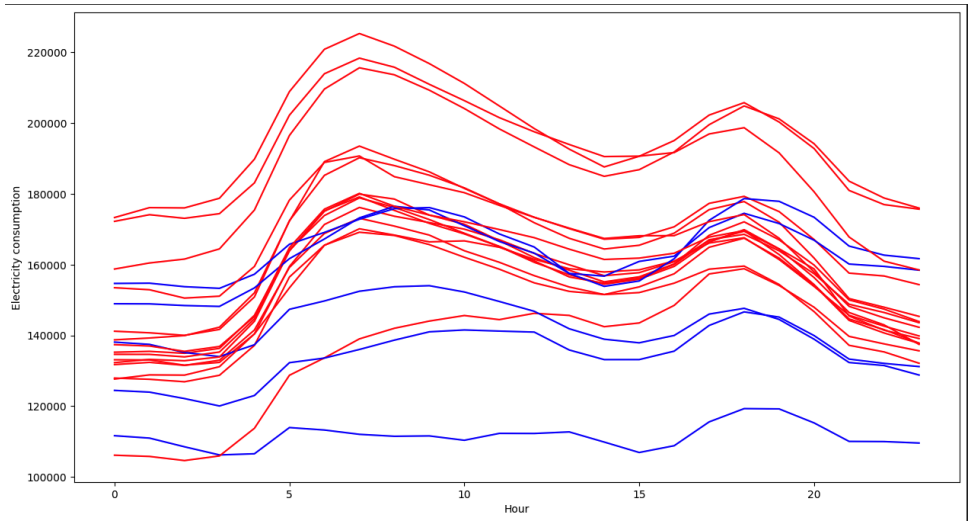

Figure: Hourly electricity consumption (one curve per day (red: week day, blue: week-end))

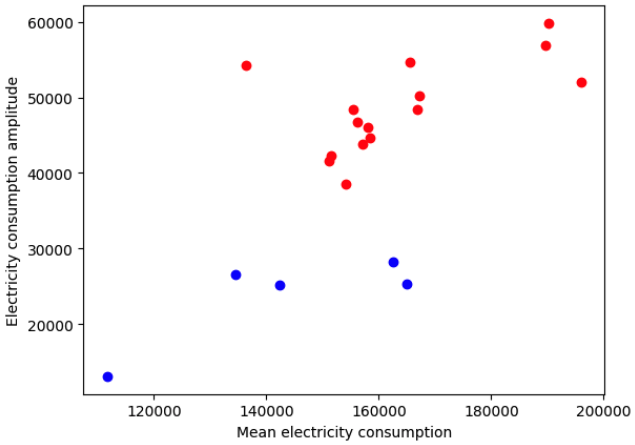

Figure: Representation after feature engineering (mean, amplitude)

Temporal features

Rolling statistics: mean, median, standard deviation, ...

- can help smooth the signal (e.g. remove high frequency / noisy variations)
- allow to incorporate aggregated past information as features (e.g. use mean, min and max temperature of the day before to predict the temperature of the next day)

Expanding statistics: cumulative sum, cumulative mean...

- computed over the whole past data (contrary to rolling statistics calculated over a specified window size)
- allow to account for cumulative quantities: e.g. number of hour of functioning for an equipment (to predict the risk of failure)

Lags: shift the time series to treat past observations as features

- account for temporal correlation for modeling (e.g. use the temperature yesterday at the same hour to predict today's temperature)

Temporal features: example

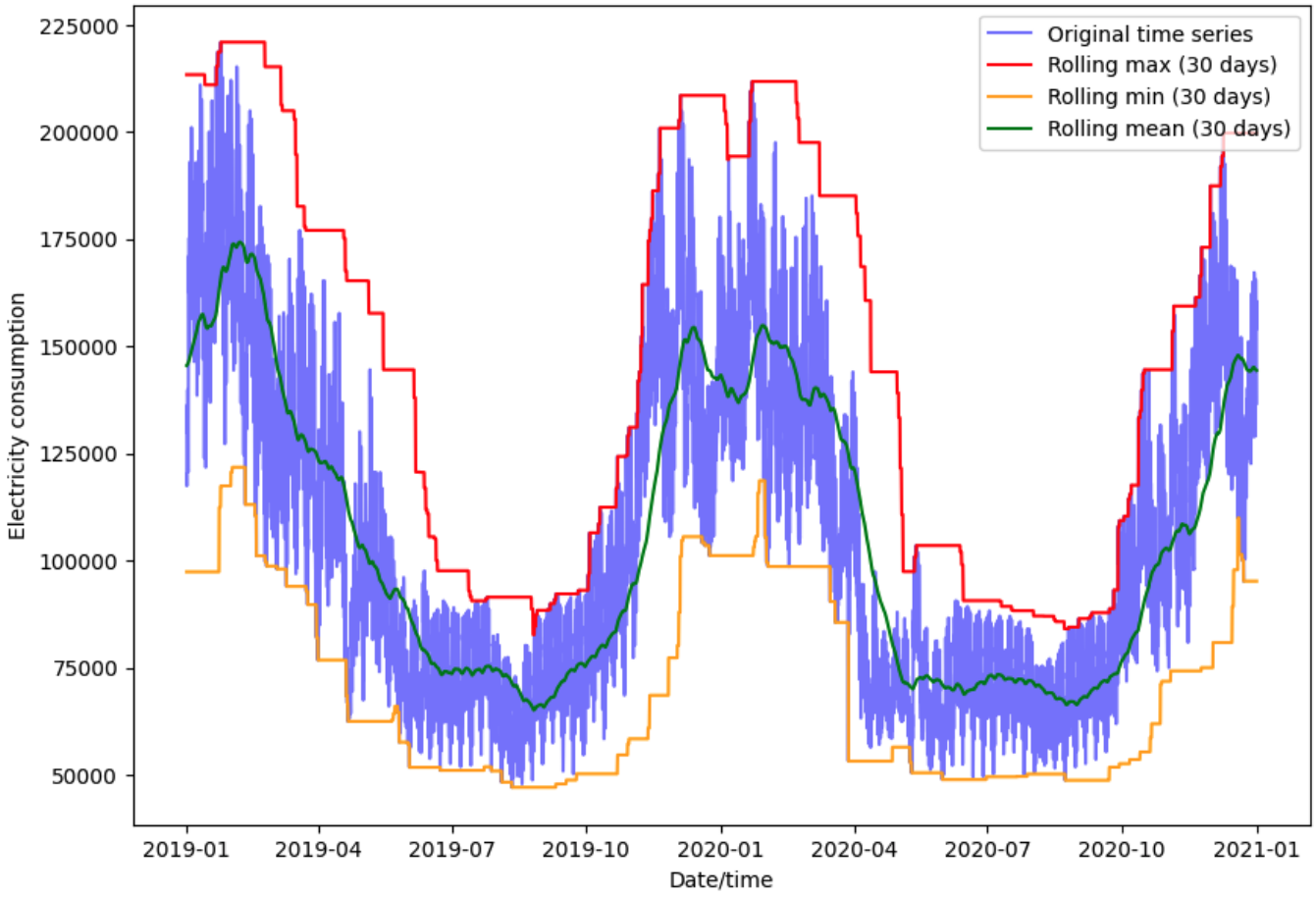

Figure: Rolling statistics (over 30-days-windows) for electricity consumption in France

Temporal features

Remarks:

- temporal features are well suited for timestamp-level tasks (forecasting, anomaly detection , change-point detection...)
- always be vigilant to only extract features that will be available at inference time: e.g. if the goal is to predict the temperature next day, do not train a model to predict the current temperature based on the temperature of the past hour !

Frequency-domain features

Principle:

- extract periodic patterns and dominant frequencies of a signal
- rely on Discrete Fourier Transform to represent signal in the frequency domain

Definition (Discrete Fourier Transform (DFT))

Let $(X_n)_{0 \leq n \leq N-1}$ be a univariate time series. The DFT of X is the complex sequence $(Y_k)_{0 \leq k \leq N-1}$ where:

$$\forall k, Y_k = \sum_{n=0}^{N-1} X_k e^{-\frac{2i\pi kn}{N}} .$$

Remarks:

- the DFT coefficient are complex values $\rightarrow$ it is common to analyze their module or squared module $(|Y_k|^2)_{0 \leq k \leq N-1}$.
- the quantity Y_k corresponds to the DFT for the frequency $\frac{k\,f}{N}$ where f is the acquisition frequency $f = \frac{1}{\delta}$

DFT - illustration

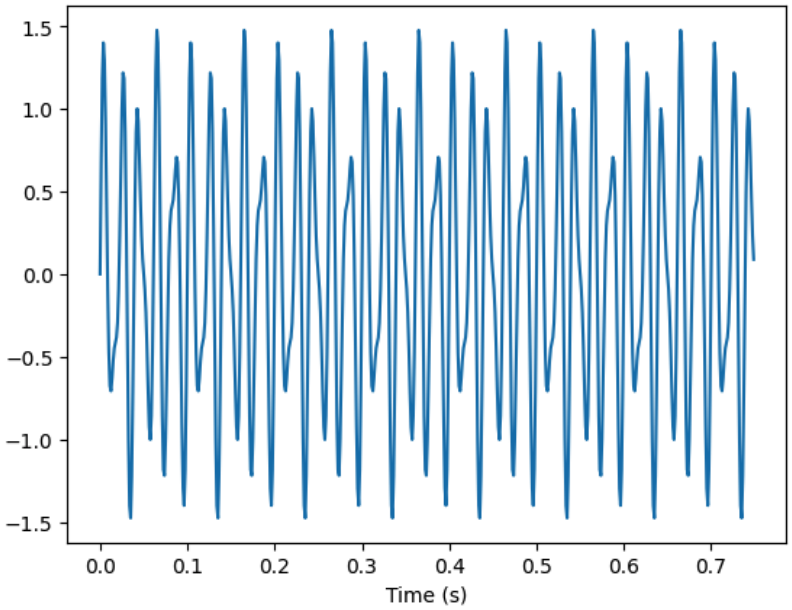

Figure: Signal in the temporal domain

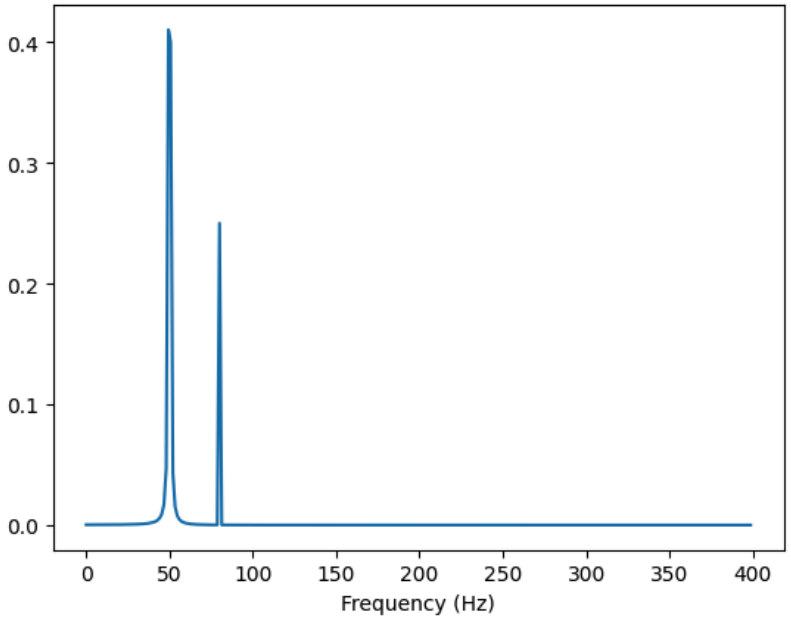

Figure: Signal in the frequency domain (after DFT transform)

Remark:

- ok as far as the spectral density does not change over time
- what if it is the case ? (e.g.: audio data...)

Spectrogram

Principle:

- apply DFT over sliding windows on the signal
- represent the signal as a 2D matrix in the time / frequency domain

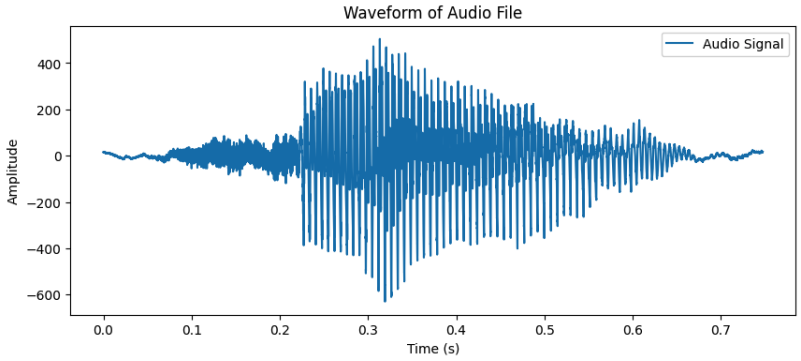

Figure: Signal in the temporal domain (audio MNIST)

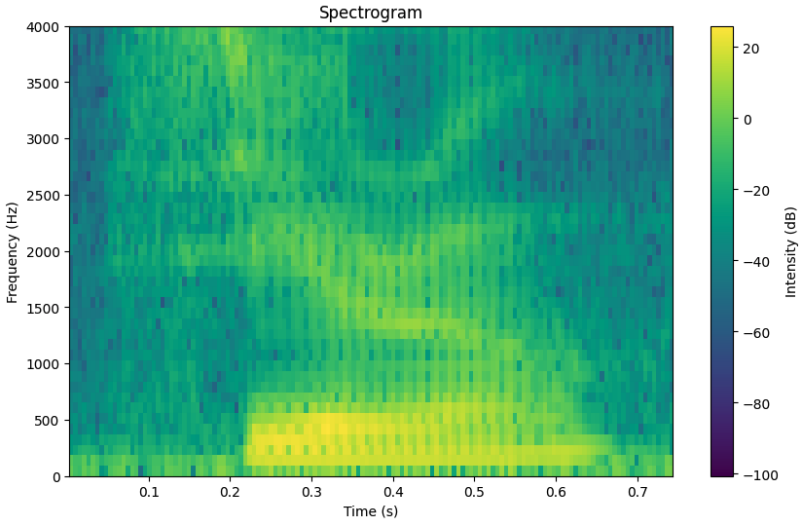

Figure: Spectrogram (audio MNIST)

To go further: time series neural representations

General principle:

- learn a relevant representation of a time series $x = (x_{1 \leq i \leq N}) \in \mathbb{R}^N$ in a low-dimensional latent space $z \in \mathbb{R}^d$
- the mapping $\Phi : x \mapsto z$ can be modeled using a neural network architecture
- the parameters of the neural network are estimated over a set of time series $(x^{(j)})_j$ so that two series close in the temporal space remain close in the latent space
- multiple structures: encoder-decoder, encoder-only, decoder-only...

A few references to go further:

- https://arxiv.org/pdf/1901.10738
- https://arxiv.org/pdf/2211.14730
- https://arxiv.org/pdf/2306.05880

Table of contents

1. Introduction

2. Key concepts in time series analysis

3. Pre-processing for time series

4. Feature extraction

5. Models for time series

6. Best practices

7. Conclusion

Models for time series

How to get back to standard machine learning:

- sequence-level tasks: e.g. classify time series or segments of time series.
  - feature engineering to extract features from time series $\rightarrow$ tabular data
  - apply standard machine learning algorithms (e.g. for classification: Logistic Regression, Random Forest...)
- timestamp-level tasks: e.g. forecasting
  - still possible to get back to "standard machine learning" after feature extraction (calendar features, lags, rolling statistics...)
  - models specifically designed to handle time series $\rightarrow$ we will focus on some of these techniques

Overview of models for time series

- Seasonal-trend decomposition (cf. preprocessing)
- Auto-regressive models (AR) and variants (ARMA, ARIMA, SARIMA...)
- Deep Learning Architectures (RNN, LSTM, GRU...)

Auto-Regressive model

Definition of the model:

$$X_t = \sum_{k=1}^{p} a_k X_{t-k} + \varepsilon_t$$

- a_1, ..., a_p are unknown coefficients
- p is an hyper-parameter
- ε is a white noise (centered signal with constant variance and null auto-covariance)

Forecasting with auto-regressive model:

- estimate the parameters (a_1, ..., a_p) using the observed time series $(X_i)_{1 \leq i \leq n}$
- use the estimated parameters to predict recursively the next value(s)
- simulate multiple trajectories with random noise to build confidence intervals

Estimation of the parameters

Assuming that the signal is stationary, the parameters must satisfy the following condition:

$$\begin{pmatrix} \gamma_1 \\ \gamma_2 \\ \cdots \\ \gamma_p \end{pmatrix} = \begin{pmatrix} \gamma_0 & \gamma_1 & \cdots & \gamma_{p-1} \\ \gamma_1 & \gamma_0 & \cdots & \gamma_{p-2} \\ \cdots & \cdots & \cdots & \cdots \\ \gamma_{p-1} & \gamma_{p-2} & \cdots & \gamma_0 \end{pmatrix} \times \begin{pmatrix} a_1 \\ a_2 \\ \cdots \\ a_p \end{pmatrix}$$

where γ denote the autocovariance function.

Estimation of parameters:

- in the above equation, replace the autocovariance by its estimate
- solve the system

Partial Auto-Correlation Function and choice of p

Partial Auto-correlation:

- let $a_k^{(p)}$ the kth regression coefficient estimated in the model of order p
- the partial auto-correlation function $PACF$ is defined as:

$$PACF(p) = a_p^{(p)}$$

- the $PACF$ can be used to select the hyper-parameter $p \rightarrow$ choose p such that $\forall k > p$, $PACF(k)$ is not significant.

ACF vs PACF

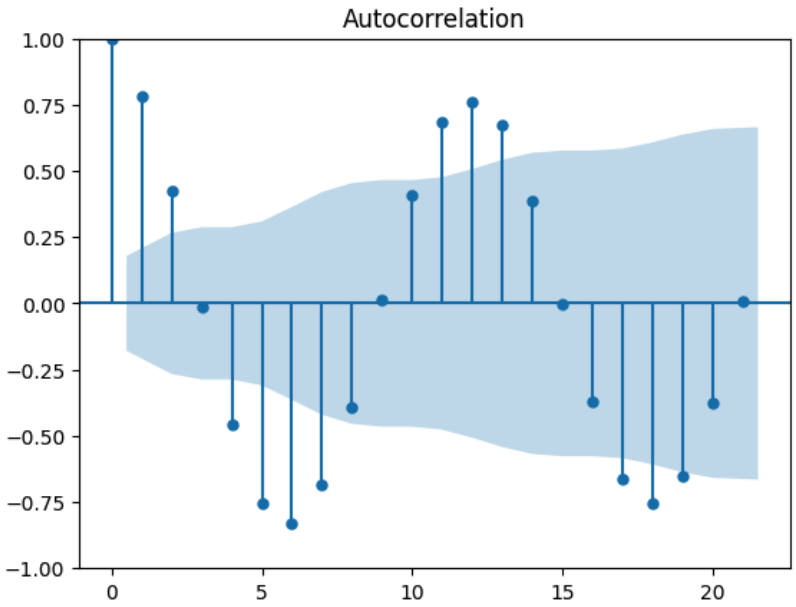

Figure: Auto-Correlation Function (monthly temperatures in Paris)

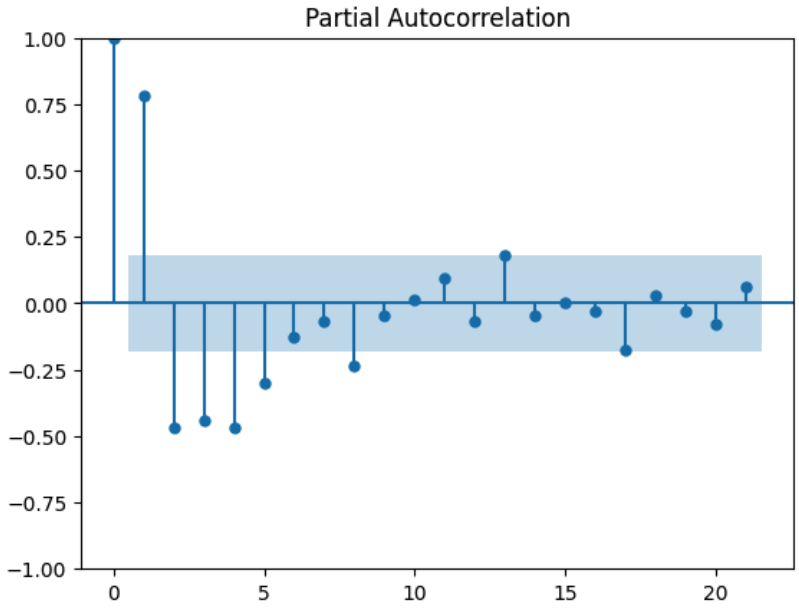

Figure: Partial Auto-Correlation Function (monthly temperatures in Paris)

Forecasting with Auto-Regressive model

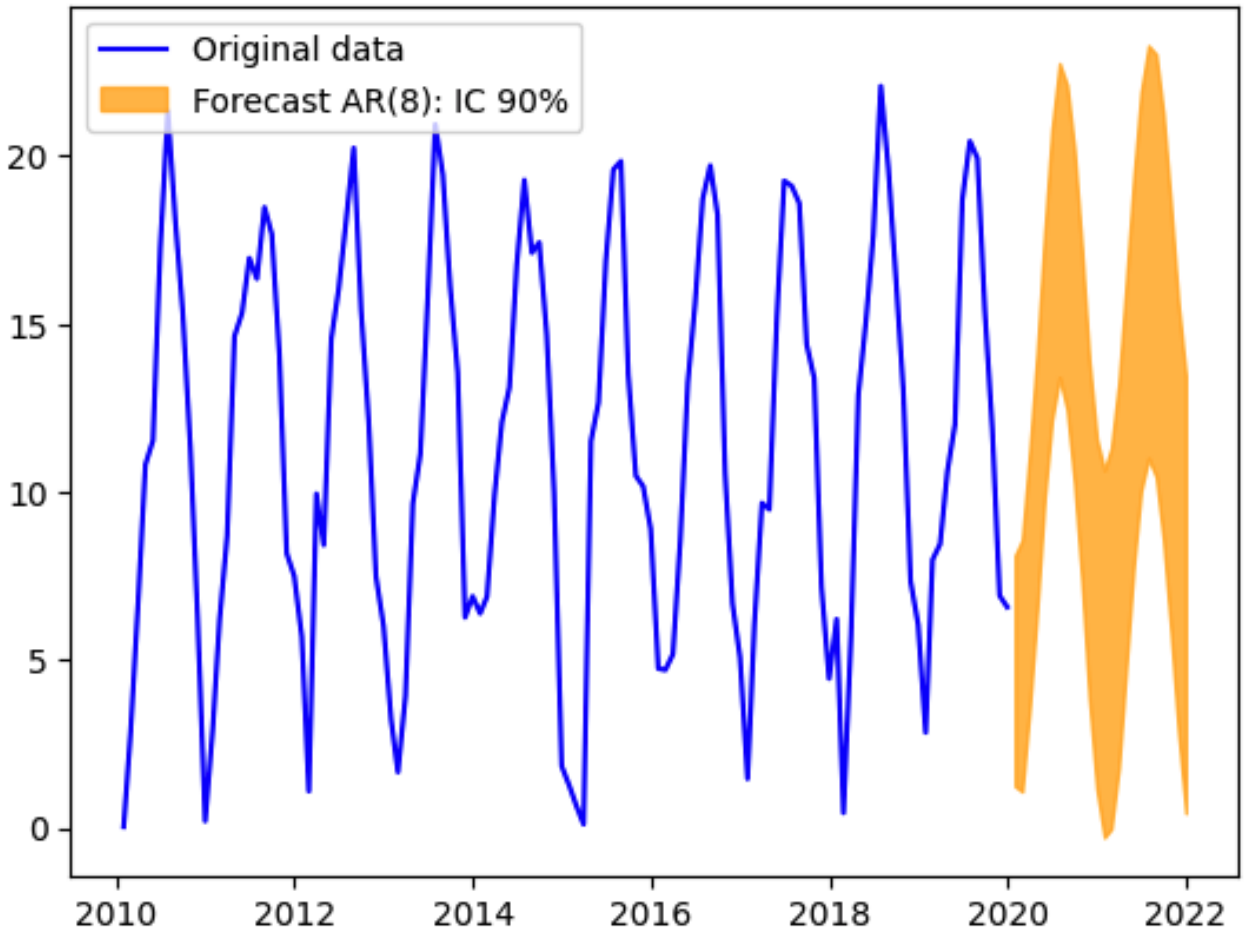

Figure: AR(8) model estimated on monthly temperatures in Paris

Moving average (MA) model

Definition of the model:

$$X_t = \varepsilon_t + \sum_{k=1}^{q} b_k \varepsilon_{t-k}$$

- $b_1,\ldots,b_q$ are unknown coefficients
- q is a hyper-parameter
- ε is a white noise

Auto-Regressive Moving-Average (ARMA) model

Definition of the model:

$$X_t = \sum_{k=1}^{p} a_k X_{t-k} + \varepsilon_t + \sum_{k=1}^{q} b_k \varepsilon_{t-k}$$

- $a_1,\ldots,a_p$ and $b_1,\ldots,b_q$ are unknown coefficients
- p and q are hyper-parameters
- ε is a white noise

Auto-Regressive Integrated Moving-Average (ARIMA)

Definition of the model:

$$X_t^{(d)} = \sum_{k=1}^{p} a_k X_{t-k} + \varepsilon_t + \sum_{k=1}^{q} b_k \varepsilon_{t-k}$$

- $X^{(d)}$ denote the dth differentiation of X
- $a_1,\ldots,a_p$ and $b_1,\ldots,b_q$ are unknown coefficients
- p, q and d are hyper-parameters
- ε is a white noise

Remarks:

- the choice of hyper-parameters and the estimation are more complex (not discussed here)
- there are other variants of these models : SARIMA, ...

Deep learning models for time series

Motivations:

- handle large datasets
- capture non-linear and complex dependencies
- "automatic" feature engineering
- handle multivariate time series

Overview of deep Learning Architectures for time series:

- Recurrent Neural Networks (RNN)
- Long-Short Term Memory (LSTM)
- Gated Recurrent Units (GRU)
- Convolutional Neural Networks (CNN)
- Transformers

Table of contents

1. Introduction
2. Key concepts in time series analysis
3. Pre-processing for time series
4. Feature extraction
5. Models for time series
6. Best practices
7. Conclusion

Importance of Proper Train-Test Split for time series

- **Avoid Random Splitting:** Random splits break temporal dependencies, introducing data leakage.
  - can lead to over-optimistic results
- **Use Temporal Splits:**
  - Train on the past, validate and test on future data.
  - Example: *Train: Jan 2020 – Dec 2021, Validate: Jan 2022 – Dec 2022, Test: Jan 2023 – Dec 2023.*
- **Time-Based Cross-Validation:** Use rolling or expanding windows for validation.

Example of improper Train-Test split

Data: temperature measurements at Eiffel Tour between 2010 and 2020.

Sampling: one observation per hour

Task: model the temperature as a function of calendar information (hour, day, month, year)

Two different Train-Test splitting strategies:

- Random Split (75% train, 25% test)
- Temporal Split (Train: Jan 2010 – Aug 2017, Test : Aug 2017 – Dec 2019)

year	month	day	hour	T_pred
2010	1	1	0	-1.3160
2010	1	1	1	-1.4038
2010	1	1	2	-1.6502
2010	1	1	3	-1.9302
2010	1	1	4	-2.1390
...	...	...	...	...
2019	12	31	19	4.7500
2019	12	31	20	4.6916
2019	12	31	21	4.4224
2019	12	31	22	4.2016
2019	12	31	23	4.5950

Figure: Overview of the dataset

Prediction after random Train-Test split

Trained model: Random Forest Regressor
Mean Absolute Error on test set: 0.658 (less than 1 degree !!!)

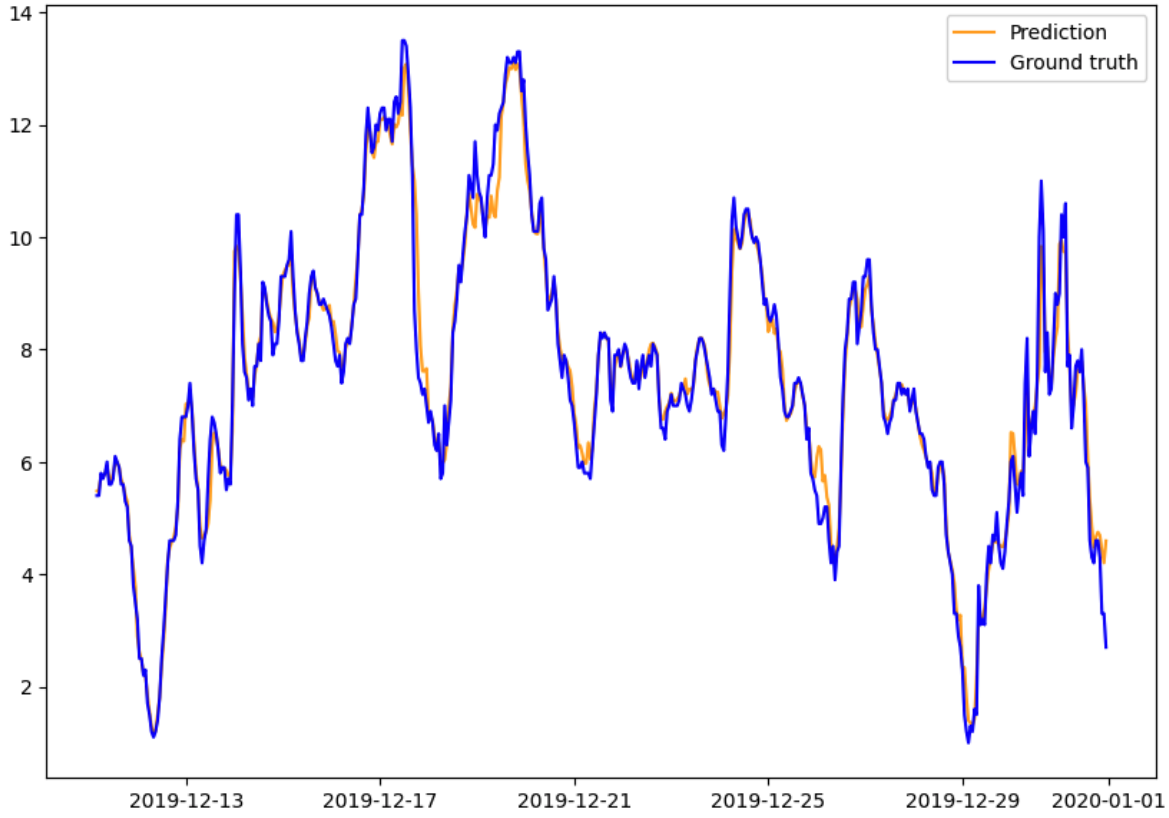

Figure: Predictions on the last 500 timestamps

Is it realistic ?

Prediction after temporal Train-Test split

Trained model: Random Forest Regressor
Mean Absolute Error on test set: 4.318

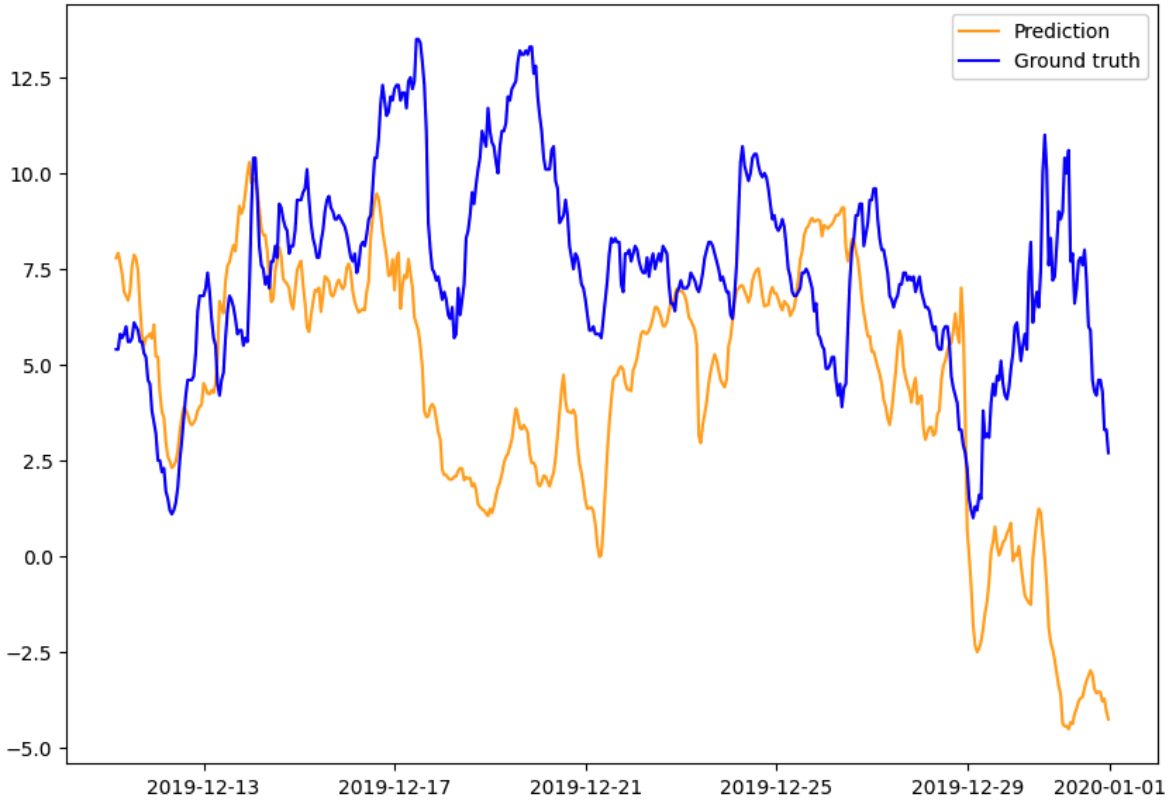

Figure: Predictions on the last 500 timestamps

The test results are significantly different and less accurate!

Example of improper Train-Test split

What happened ?

- With a random train-test split:
  - The test instances are spread over the whole timeline
  - It is thus likely that for each timestamp in the test set, there is a close timestamp in the train set (one or a few hours apart)
  - A model that can predict well on train set will predict accurately for observation near train examples
  - There is a strong dependence between train and test examples
  - Thus, the overfitting problem remains undetected...
- With a temporal train-test split:
  - We can reasonably assume that the test set is independent from the training set (not really the case though...)
  - The predictions on the test set allow us to estimate more accurately how the model generalizes to unseen and independent data

$\Rightarrow$ **The splitting strategy MUST preserve the chronological order.**

Proper Train-Test Splitting strategies

A few proper Splitting strategies:

- Temporal Train-Test split
- Sliding Window Splitter
- Expanding Window Splitter

Remark

- Each of the above strategies preserve chronological order
- Sliding Window Splitter and Expanding Window splitter are well suited for evaluating forecasting models

Sliding Window Splitter

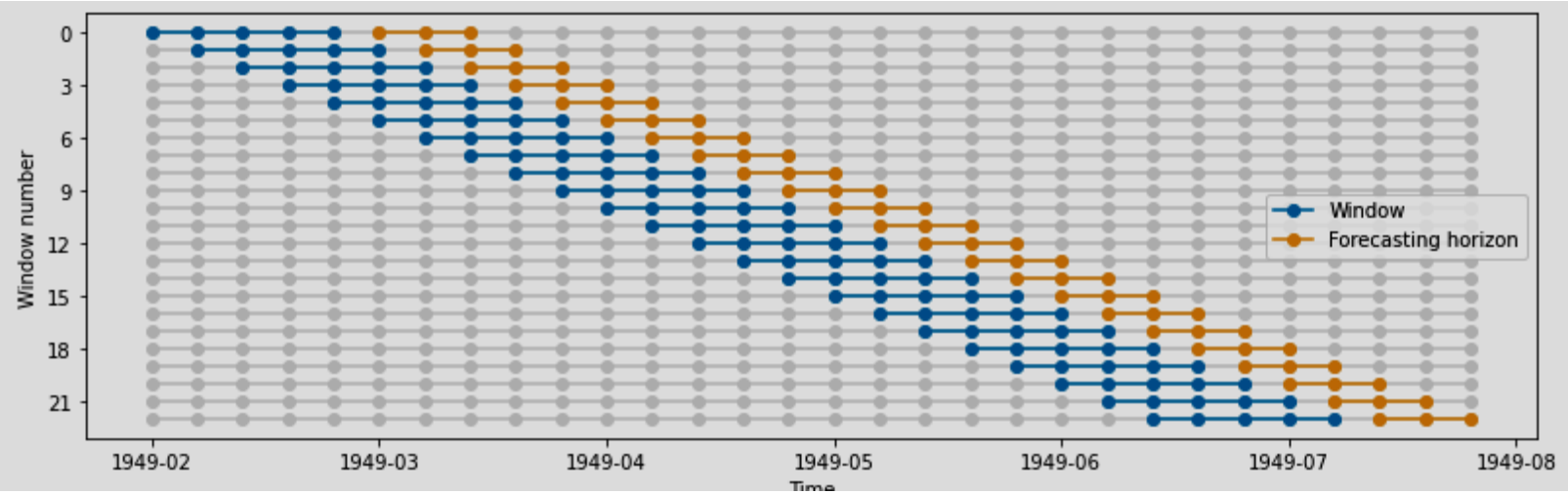

Figure: Sliding Window Splitter *(sktime documentation)*

Expanding Window Splitter

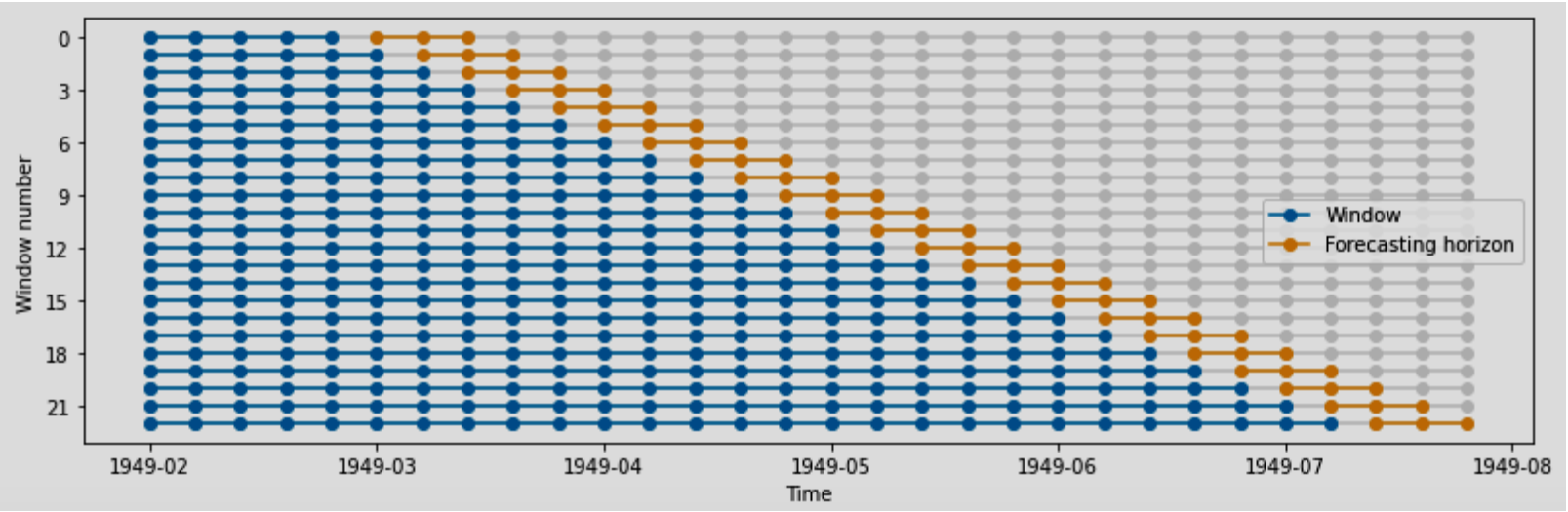

Figure: Expanding Window Splitter *(sktime documentation)*

Naive forecasting as a baseline

Always compare forecasting models with naive methods:

- **Mean Forecast:** Predict the average value.
- **Last Observation:** Repeat the last observed value.

Benefits of naive baselines:

- Simple yet effective for many time series.
- Provides a benchmark to measure model improvements.

Example: Forecasting $\widehat{y}_t = y_{t-1}$.

Naive forecasting as a baseline: example

Task: Predict the temperature of next hour based on present and past information

Features extracted: temperature values for the 6 previous hours

Fitted model: linear regression

- Test R2 score: 0.991 $\rightarrow$ excellent ?

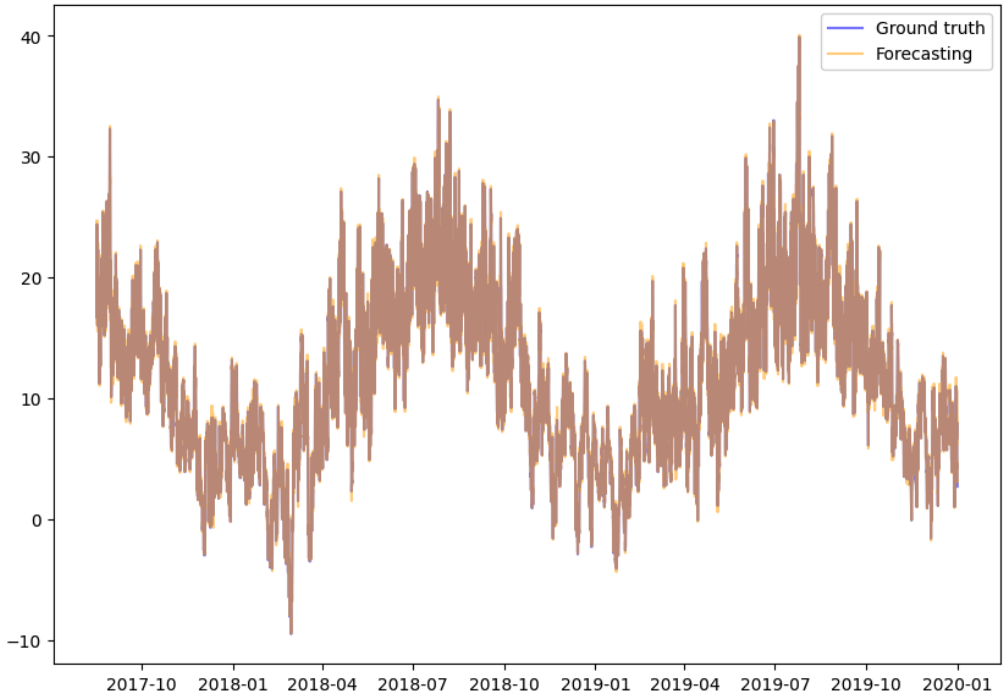

Figure: Predictions on test set

Naive forecasting as a baseline: example

In fact, linear regression is not significantly better than naive prediction...

- Test R2 score: 0.991 $\rightarrow$ excellent ?
- Test R2 score for naive forecaster (repeat previous value): 0.989.

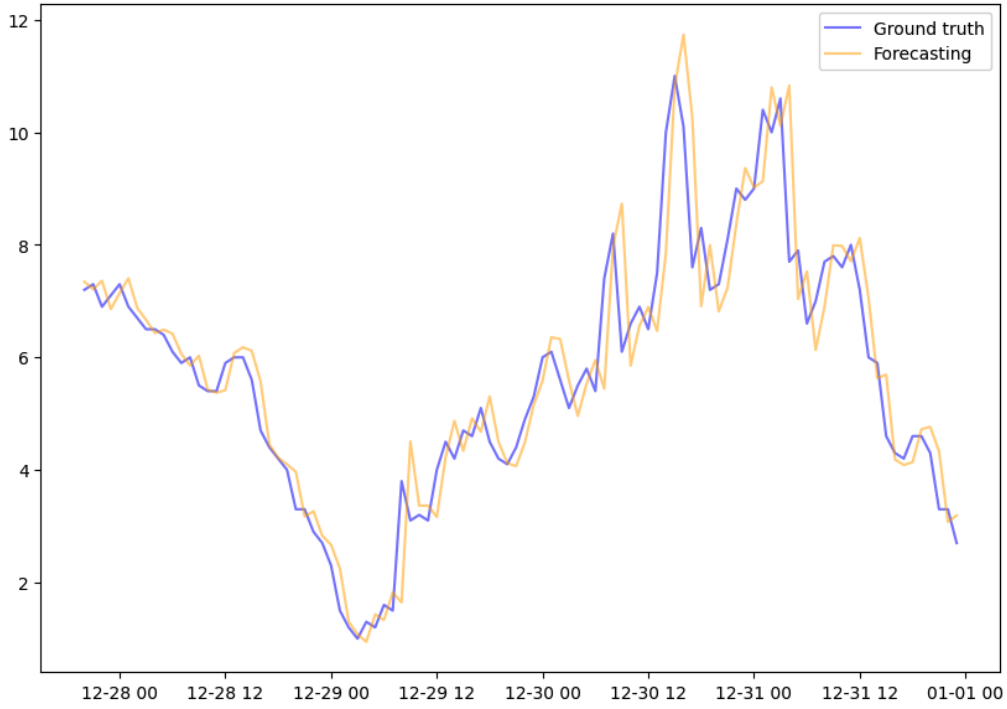

Figure: Predictions on test set

$\Rightarrow$ always compare performances to relevant baselines!

Table of contents

1. Introduction
2. Key concepts in time series analysis
3. Pre-processing for time series
4. Feature extraction
5. Models for time series
6. Best practices
7. Conclusion

Conclusion

From time series to tabular data

- Always possible to transform time series into standard tabular data through feature extraction (e.g., statistical features, FFT, lag features).

Specialized techniques

- Time series problems can benefit from modeling techniques specifically designed for temporal data (e.g., ARIMA, RNN...).

Beware of Temporal Dependencies

- Temporal dependency is critical; avoid data leakage by carefully splitting data (e.g., time-based splits instead of random splits).

Naive Baselines are Essential

- Always compare forecasting models to simple baselines (e.g., last observed value, mean forecast) to validate model effectiveness.

Information Availability

- Ensure that only information available at the time of prediction is included in the model to maintain validity during testing.